

দিন ১

Complexity, Array ও Two Pointer

সব কিছুর ভিত্তি — আজকের দিনটা ঠিকমতো বুঝলে বাকি ৬ দিন সহজ হয়ে যাবে

আজ যা শিখবো

- প্রবলেম সলভিং ইন্টারভিউ কীভাবে হয় এবং ৬-ধাপের উত্তর দেওয়ার ফর্মুলা
- Time Complexity ও Big-O — হিসাব করার নিয়ম
- Space Complexity
- Array — মেমোরিতে কীভাবে থাকে, কোন অপারেশনের খরচ কত
- Two Pointer প্যাটার্ন
- ৫টি গুরুত্বপূর্ণ কোডিং প্রশ্ন (PHP সহ)

কোর্স ভিডিও: ১-৭, ১৪

দিন শেষে তুমি পারবে

- যেকোনো কোড দেখে তার Time ও Space Complexity বলতে পারবে
- Brute force থেকে optimized সলিউশনে যাওয়ার চিন্তা করতে পারবে
- Two Sum, Reverse, Stock, Move Zeroes, Remove Duplicates নিজে লিখতে পারবে

স্ট্যান্ডি শিডিউল (প্রায় ৫-৬ ঘণ্টা)

১ ঘণ্টা	কনসেপ্ট অংশ পড়ো (Big-O, Array, Two Pointer)
২.৫ ঘণ্টা	৫টি প্রশ্ন — আগে নিজে চেষ্টা, তারপর সলিউশন
৩০ মিনিট	খিওরি প্রশ্নোত্তর জোরে জোরে বলো
১ ঘণ্টা	প্র্যাকটিস প্রবলেম থেকে অন্তত ২টা
১৫ মিনিট	চিট শিট রিভিশন

অধ্যায় ১: প্রবলেম সলভিং ইন্টারভিউ আসলে কী?

ইন্টারভিউয়ার শুধু দেখে না তোমার কোড চলে কি না — দেখে তুমি **কীভাবে চিন্তা করো**। চুপচাপ কোড লিখে ফেলার চেয়ে নিজের চিন্তা মুখে বলতে বলতে সলভ করা অনেক বেশি নম্বর পায়। একটা ভুল সলিউশনও ভালো কমিউনিকেশনের কারণে পাস করতে পারে।

৬-ধাপের উত্তর দেওয়ার ফর্মুলা

ধাপ	কী করবে	কী বলবে (উদাহরণ)
১. প্রশ্ন বোঝো	ইনপুট, আউটপুট, শর্ত নিশ্চিত করো	"অ্যারে কি sorted? খালি অ্যারে আসতে পারে? নেগেটিভ নাম্বার আছে?"
২. উদাহরণ নাও	ছোট একটা উদাহরণ হাতে করে সলভ করো	"ধরি [2,7,11], target 9 — উত্তর হবে index 0 আর 1"
৩. Brute force	সবচেয়ে সহজ (ধীর হলেও) উপায় বলো	"সহজ উপায় হলো দুইটা লুপ — এটা $O(n^2)$ "
৪. Optimize	কোথায় বারবার একই কাজ হচ্ছে খোঁজো	"hash map দিয়ে আগের দেখা মান মনে রাখলে $O(n)$ হবে"
৫. কোড লেখো	পরিষ্কার নাম দিয়ে কোড লেখো	লেখার সময় প্রতিটা লাইনের উদ্দেশ্য বলো
৬. টেস্ট ও Complexity	উদাহরণ দিয়ে ড্রাই রান + এজ কেস	"Time $O(n)$, Space $O(n)$ । খালি অ্যারেতে [] রিটার্ন করবে"

মনে রাখো

Brute force বলতে লজ্জা পাবে না। ইন্টারভিউয়ার আসলে এটাই শুনতে চায় — এরপর তুমি কীভাবে উন্নত করো সেটাই আসল পরীক্ষা।

সাধারণ ভুল

- প্রশ্ন পুরো না বুঝে কোড শুরু করা
- ২-৩ মিনিট চুপ করে থাকা — আটকে গেলেও জোরে চিন্তা করো
- কোড শেষে টেস্ট না করা
- Complexity জিজ্ঞেস করার আগেই নিজে না বলা

অধ্যায় ২: Time Complexity ও Big-O

Time Complexity মানে সেকেন্ড গোনো না। এটা বলে: **ইনপুট (n) বড় হলে কাজের পরিমাণ কী হারে বাড়ে**। কারণ একই কোড ধীর কম্পিউটারে ধীর, দ্রুত কম্পিউটারে দ্রুত চলে — তাই আমরা "ধাপের সংখ্যা" গুনি।

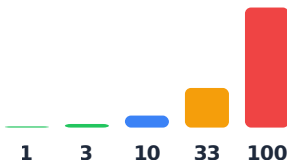
বাস্তব উদাহরণ: বইয়ের তাকে বই খোঁজা

- **$O(1)$** — তুমি জানো বইটা ৫ নম্বর তাকে। সরাসরি হাত দিলে। বই ১০টা হোক বা ১০ লাখ — সময় একই।
- **$O(n)$** — বই এলোমেলো, একটা একটা করে দেখছো। বই দ্বিগুণ হলে সময়ও দ্বিগুণ।
- **$O(\log n)$** — বই নাম অনুযায়ী সাজানো। মাঝখান খুলে দেখো, অর্ধেক বাদ দাও — বারবার। ১০ লাখ বইয়েও মাত্র ~২০ বার দেখা লাগে!
- **$O(n^2)$** — প্রতিটা বইকে বাকি সব বইয়ের সাথে তুলনা করছো (যেমন ডুপ্লিকেট খোঁজা)।

কোনটা কত দ্রুত বাড়ে? (n = ইনপুটের সাইজ)

Complexity	নাম	n = 10	n = 1,000	n = 10,00,000	রায়
$O(1)$	Constant	1	1	1	চমৎকার
$O(\log n)$	Logarithmic	~3	~10	~20	চমৎকার
$O(n)$	Linear	10	1,000	10,00,000	ভালো
$O(n \log n)$	Linearithmic	~33	~10,000	~2 কোটি	ঠিক আছে
$O(n^2)$	Quadratic	100	10 লাখ	10^{12} (!)	বড় ইনপুটে খারাপ
$O(2^n)$	Exponential	1,024	অসম্ভব	অসম্ভব	খুব খারাপ

n = 10 হলে কাজের পরিমাণ: $O(1) \cdot O(\log n) \cdot O(n) \cdot O(n \log n) \cdot O(n^2)$



ইন্টারভিউ রুল অফ থাম্ব

সাধারণত ১ সেকেন্ডে $\sim 10^8$ সাধারণ অপারেশন হয়। n = 10^5 হলে $O(n^2) = 10^{10}$ — টাইম লিমিট পার! তাই n বড় (10^4 এর বেশি) হলে $O(n)$ বা $O(n \log n)$ সলিউশন খোঁজো।

Complexity হিসাব করার ৫টি নিয়ম

নিয়ম ১: একটা লুপ n বার চললে $\rightarrow O(n)$

PHP

```
function sumAll(array $nums): int {
    $sum = 0;           // ১ বার
    foreach ($nums as $x) { // n বার
        $sum += $x;      // প্রতিবার ১টা কাজ
    }
    return $sum;        // ১ বার
}
echo sumAll([1, 2, 3, 4]); // 10
// মোট = 1 + n + 1 = n + 2  $\rightarrow O(n)$ 
```

নিয়ম ২: Constant আর ছোট টার্ম বাদ দাও

$O(2n + 5) \rightarrow O(n)$ | $O(n^2 + n) \rightarrow O(n^2)$ | কারণ n খুব বড় হলে ছোট অংশগুলোর প্রভাব নগণ্য।

নিয়ম ৩: Nested লুপ $\rightarrow$ গুণ হয়

PHP

```
function printPairs(array $nums): void {
    $n = count($nums);
    for ($i = 0; $i < $n; $i++) { // n বার
        for ($j = 0; $j < $n; $j++) { // প্রতিবার আবার n বার
            echo "({$nums[$i]},{$nums[$j]}) ";
        }
    }
}
printPairs([1, 2]); // (1,1) (1,2) (2,1) (2,2)
//  $n \times n = n^2 \rightarrow O(n^2)$ 
```

নিয়ম ৪: পরপর (আলাদা) লুপ $\rightarrow$ যোগ হয়

একটা লুপ n বার, তারপর আরেকটা লুপ m বার চললে = $O(n + m)$ | দুটোই n হলে $O(2n) = O(n)$ ।

নিয়ম ৫: প্রতিবার অর্ধেক হলে $\rightarrow O(\log n)$

PHP

```
function halvingSteps(int $n): int {
    $steps = 0;
    while ($n > 1) {
        $n = intdiv($n, 2); // প্রতিবার অর্ধেক
        $steps++;
    }
    return $steps;
}
echo halvingSteps(16); // 4 (16→8→4→2→1)
echo "\n" . halvingSteps(1000000); // 19
```

দ্রুত চেনার উপায়

কোডে যা দেখবে	Complexity
লুপ নেই, শুধু হিসাব / index দিয়ে access	$O(1)$
প্রতিবার অর্ধেক করা (binary search)	$O(\log n)$
একটা লুপ	$O(n)$
ভালো sorting (merge sort, PHP sort())	$O(n \log n)$
লুপের ভেতরে লুপ	$O(n^2)$
প্রতিটা কল নিজেকে ২ বার কল করে (সাধারণ fibonacci)	$O(2^n)$

Best, Average, Worst Case

একটা অ্যারেতে ৭ খুঁজছো। প্রথম ঘরেই পেলে — **Best case $O(1)$** । শেষ ঘরে বা নেই — **Worst case $O(n)$** । ইন্টারভিউতে সাধারণত **Worst case** বলবে, কারণ এটা গ্যারান্টি দেয় "এর চেয়ে খারাপ হবে না"।

অধ্যায় ৩: Space Complexity

Space Complexity = অ্যালগরিদম চালাতে **অতিরিক্ত** কতটুকু মেমোরি লাগে (ইনপুট বাদে)।

কোড	অতিরিক্ত মেমোরি	Space
কয়েকটা ভেরিয়েবল (\$sum, \$i, \$max)	ইনপুট যত বড়ই হোক, একই	$O(1)$
নতুন অ্যারে যেটা n সাইজের হতে পারে	ইনপুটের সাথে বাড়ে	$O(n)$
$n \times n$ সাইজের 2D অ্যারে	ইনপুটের বর্গ	$O(n^2)$
Recursion যেটা n গভীরে যায়	প্রতিটা কল স্ট্যাকে জায়গা নেয়	$O(n)$

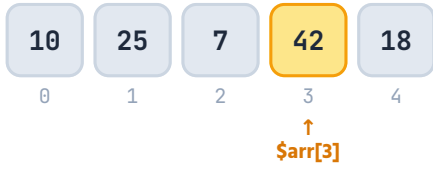
Time vs Space ট্রেড-অফ

প্রায়ই আমরা একটু বেশি মেমোরি খরচ করে অনেক সময় বাঁচাই। যেমন Two Sum-এ hash map ($O(n)$ space) নিয়ে $O(n^2)$ থেকে $O(n)$ time-এ নামিয়ে আনি। মোবাইলে ফোন নম্বর মুখস্থ না রেখে কন্টাক্ট লিস্ট সেভ রাখা — জায়গা লাগে, কিন্তু খোঁজা দ্রুত।

অধ্যায় 8: Array

Array হলো পাশাপাশি সাজানো ঘরের সারি। প্রতিটা ঘরের একটা নম্বর (**index**) আছে, যেটা 0 থেকে শুরু হয়।

arr = [10, 25, 7, 42, 18] — index 3-এ সরাসরি যাওয়া যায়



Index দিয়ে access কেন $O(1)$?

Traditional array-তে ঘরগুলো মেমোরিতে **পাশাপাশি (contiguous)** থাকে। প্রথম ঘরের ঠিকানা জানলে যেকোনো ঘরের ঠিকানা একটা গুণ-যোগেই বের হয়: $\text{address} = \text{start} + \text{index} \times \text{size}$ । তাই ১০ লাখতম ঘরেও এক ধাপে যাওয়া যায়।

Array অপারেশনের খরচ

অপারেশন	Complexity	কেন?
Access: $\$arr[i]$	$O(1)$	সরাসরি ঠিকানা হিসাব
শেষে যোগ: $\$arr[] = x$	$O(1)^*$	শেষে জায়গা থাকলে সরাসরি বসে (*amortized)
শেষ থেকে বাদ: $\text{array_pop}()$	$O(1)$	কাউকে সরাতে হয় না
শুরুতে/মাঝে যোগ বা বাদ	$O(n)$	বাকি সব ঘর এক ঘর করে সরাতে হয়
মান খোঁজা (sorted না)	$O(n)$	একটা একটা করে দেখতে হয়
মান খোঁজা (sorted)	$O(\log n)$	Binary search (দিন ২)

ধাপ 1 মাঝে (index 1-এ) 99 ঢোকাতে চাই:



ধাপ 2 25, 7, 42 — সবাইকে এক ঘর ডানে সরাতে হলো (এই সরানোটাই $O(n)$):



ধাপ 3 এখন ফাঁকা জায়গায় 99 বসালাম:



PHP-এর array একটু আলাদা!

PHP-এর array আসলে একটা **ordered hash map** — এটা একসাথে list আর dictionary দুইভাবেই কাজ করে (`$a[0]` আবার `$a["name"]`)। ইন্টারভিউতে বলবে: "PHP array ভেতরে hash table, তবে আমি এটাকে সাধারণ array হিসেবে ব্যবহার করছি।" মনে রাখো: `array_shift()` আর `array_unshift()` $O(n)$, কারণ সব index নতুন করে সাজাতে হয়।

PHP-তে Array-র দরকারি ফাংশন

ফাংশন	কাজ	Complexity
<code>count(\$a)</code>	সাইজ	$O(1)$
<code>\$a[] = \$x / array_push</code>	শেষে যোগ	$O(1)$
<code>array_pop(\$a)</code>	শেষ থেকে বাদ	$O(1)$
<code>array_shift / array_unshift</code>	শুরু থেকে বাদ / শুরুতে যোগ	$O(n)$
<code>in_array(\$x, \$a)</code>	মান আছে কি না	$O(n)$
<code>isset(\$a[\$key])</code>	key আছে কি না	$O(1)$
<code>sort(\$a)</code>	সাজানো	$O(n \log n)$
<code>array_reverse(\$a)</code>	উল্টানো (নতুন array)	$O(n)$

অধ্যায় ৫: Two Pointer প্র্যাটর্ন

Two Pointer মানে দুইটা index ভেরিয়েবল নিয়ে একসাথে অ্যাারেতে হাঁটা। এতে অনেক সময় nested লুপ ($O(n^2)$) লাগে না — এক পাসেই ($O(n)$) কাজ হয়ে যায়।

ধরন ১: দুই মাথা থেকে ভেতরের দিকে (Opposite ends)

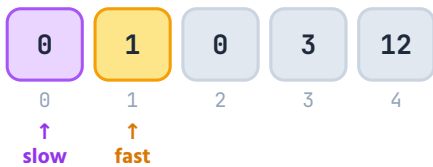
L শুরু থেকে ডানে, R শেষ থেকে বামে — দেখা হলে থামো



কখন: অ্যারে উল্টানো, palindrome চেক, sorted অ্যারেতে দুইটা সংখ্যার যোগফল খোঁজা।

ধরন ২: একই দিকে, দুই গতিতে (Slow & Fast)

fast সবকিছু দেখে, slow শুধু "ভালো" জিনিস রাখার জায়গা দেখায়



কখন: ইন-প্লেস ফিল্টার করা — শূন্য সরানো, ডুপ্লিকেট বাদ দেওয়া, linked list-এর মাঝখান খোঁজা।

চেনার সংকেত

প্রশ্নে যদি থাকে: **"sorted array"**, **"in-place"**, **"extra space ব্যবহার করো না"**, **"pair খোঁজো"** — তাহলে প্রথমেই Two Pointer ভাবো।

প্রশ্ন 1.1

Two Sum

দুইটা সংখ্যা খোঁজো যাদের যোগফল target

Easy

LeetCode 1

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

সবচেয়ে বেশি জিজ্ঞেস করা ইন্টারভিউ প্রশ্ন। এটা শেখায় কীভাবে hash map দিয়ে $O(n^2)$ থেকে $O(n)$ -এ নামা যায় — এই কৌশল শত শত প্রশ্নে লাগে।

সমস্যাটা কী?

একটা integer array `nums` আর একটা `target` দেওয়া আছে। এমন দুইটা সংখ্যার **index** রিটার্ন করো যাদের যোগফল target। ধরে নাও ঠিক একটা উত্তরই আছে, আর একই element দুইবার ব্যবহার করা যাবে না।

উদাহরণ

Input: `nums = [2, 7, 11, 15]`, `target = 9`
 Output: `[0, 1]` // কারণ $2 + 7 = 9$
 Input: `nums = [3, 2, 4]`, `target = 6`
 Output: `[1, 2]` // $2 + 4 = 6$ (3+3 নয়, একই element দুইবার না)

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

তুমি দোকানে ৯০০ টাকার গিফট ভাউচার নিয়ে গেছো, ঠিক দুইটা জিনিস কিনতে চাও। প্রতিটা জিনিস দেখে মনে মনে ভাবো "এটা ৩০০ হলে আমার আরেকটা ৬০০-র জিনিস দরকার।" আগে কোনো ৬০০ টাকার জিনিস দেখে থাকলে — উত্তর পেয়ে গেছো! এই "আগে দেখেছি কি না" মনে রাখার খাতাটাই hash map।

পদ্ধতি ১: Brute Force — সব জোড়া চেক করো

- প্রতিটা i -এর জন্য, তার পরের প্রতিটা j দেখো
- `nums[i] + nums[j] == target` হলে `[i, j]` রিটার্ন

PHP

```
function twoSumBrute(array $nums, int $target): array {
    $n = count($nums);
    for ($i = 0; $i < $n; $i++) {
        for ($j = $i + 1; $j < $n; $j++) { // i-এর পর থেকে, যাতে একই element না হয়
            if ($nums[$i] + $nums[$j] === $target) {
                return [$i, $j];
            }
        }
    }
    return []; // উত্তর না পেলে
}

print_r(twoSumBrute([2, 7, 11, 15], 9)); // [0, 1]
```

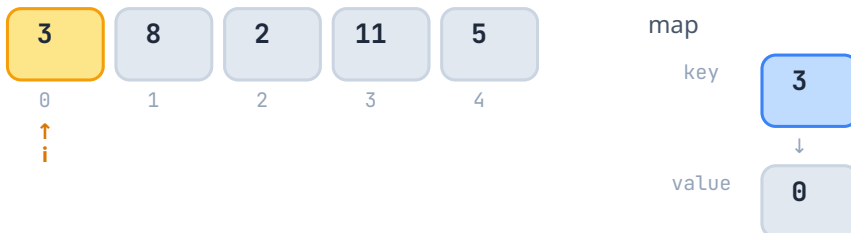
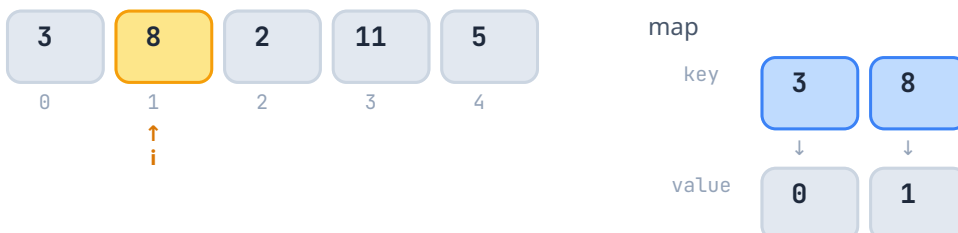
Time: $O(n^2)$ Space: $O(1)$ সমস্যা: $n = 10,000$ হলে প্রায় ৫ কোটি জোড়া চেক করতে হয়।**পদ্ধতি ২: Optimized — Hash Map (এক পাসে)**

- একটা খালি map রাখো: মান $\rightarrow$ index
- প্রতিটা সংখ্যার জন্য হিসাব করো $need = target - num$
- need যদি map-এ থাকে $\rightarrow$ উত্তর পেয়ে গেছো
- না থাকলে বর্তমান সংখ্যাটা map-এ রেখে দাও

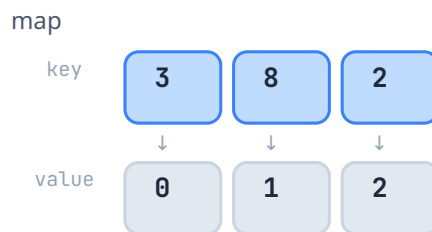
PHP

```
function twoSum(array $nums, int $target): array {
    $seen = []; // মান => index
    foreach ($nums as $i => $num) {
        $need = $target - $num; // আমার জোড়া কত হওয়া দরকার?
        if (isset($seen[$need])) { // আগে দেখেছি?  $O(1)$  চেক
            return [$seen[$need], $i];
        }
        $seen[$num] = $i; // পরের জনদের জন্য মনে রাখি
    }
    return [];
}

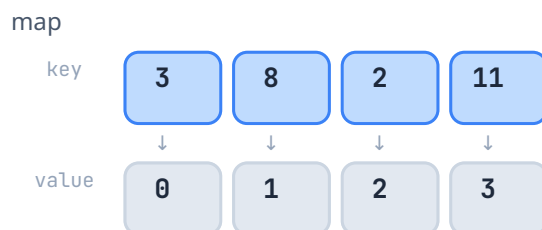
print_r(twoSum([2, 7, 11, 15], 9)); // [0, 1]
print_r(twoSum([3, 2, 4], 6)); // [1, 2]
```

Time: $O(n)$ Space: $O(n)$ কেন আগে চেক, পরে রাখি? যাতে একই element নিজের সাথেই জোড়া না বানায় (যেমন $[3, 2, 4]$, target 6-এ $3+3$)।**ড্রাই রান — ধাপে ধাপে দেখি****ধাপ 1** $nums = [3, 8, 2, 11, 5]$, target = 7। $i=0$, $num=3 \rightarrow need=4$ । map-এ 4 নেই। 3 রাখলাম।**ধাপ 2** $i=1$, $num=8 \rightarrow need=-1$ । নেই। 8 রাখলাম।

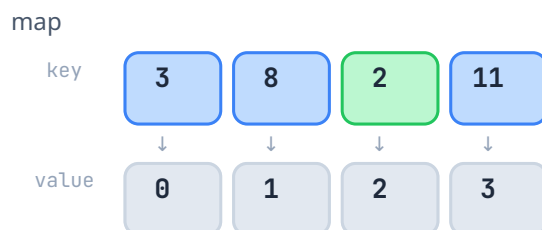
ধাপ 3 $i=2$, $num=2 \rightarrow need=5$ । নেই। 2 রাখলাম।



ধাপ 4 $i=3$, $num=11 \rightarrow need=-4$ । নেই। 11 রাখলাম।



ধাপ 5 $i=4$, $num=5 \rightarrow need=2$ । **map-এ 2 আছে, index 2!** উত্তর = [2, 4]



এজ কেস (এগুলো ভুলবে না)

- ডুপ্লিকেট মান: [3, 3], target 6 $\rightarrow$ [0, 1] (আমাদের কোড ঠিক কাজ করে, কারণ আগে চেক করি)
- নেগেটিভ সংখ্যা: [-1, -2, -3], target -5 $\rightarrow$ কোনো সমস্যা নেই
- উত্তর না থাকলে খালি array রিটার্ন

ইন্টারভিউতে যা বলবে

- প্রথমে brute force বলো, তারপর বলো: "ভেতরের লুপটা আসলে একটা মান খুঁজছে — খোঁজাটা $O(1)$ করতে পারলে পুরোটা $O(n)$ হবে।"
- Trade-off বলো: " $O(n)$ extra space নিয়ে time $O(n^2)$ থেকে $O(n)$ -এ নামালাম।"

ফলো-আপ প্রশ্ন (ইন্টারভিউয়ার জিজ্ঞেস করতে পারে)

"অ্যারে যদি **sorted** হয়?" তখন Two Pointer: $L=0$, R =শেষ। যোগফল ছোট হলে $L++$, বড় হলে $R--$ । Space $O(1)$!
(LeetCode 167 — প্র্যাকটিসে আছে)

প্রশ্ন 1.2

Reverse an Array / String

অ্যারে বা স্ট্রিং উল্টে দাও (built-in ছাড়া)

Easy

LeetCode 344

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

Two Pointer শেখার সবচেয়ে সহজ প্রশ্ন। ইন্টারভিউতে প্রায়ই বলে "array_reverse / strrev ব্যবহার করা যাবে না।"

সমস্যাটা কী?

একটা array (বা string) দেওয়া আছে। এটাকে **in-place** উল্টে দাও — মানে নতুন array না বানিয়ে একই array পরিবর্তন করো।

উদাহরণ

Input: [1, 2, 3, 4, 5]

Output: [5, 4, 3, 2, 1]

Input: "hello"

Output: "olleh"

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

লাইনে দাঁড়ানো ৫ জন মানুষকে উল্টো ক্রমে সাজাতে বলা হলো। সবচেয়ে সহজ: প্রথম আর শেষ জন জায়গা বদলাও, তারপর দ্বিতীয় আর শেষ থেকে দ্বিতীয় জন... মাঝখানে পৌঁছালে কাজ শেষ।

পদ্ধতি ১: নতুন array-তে পেছন থেকে কপি

- একটা খালি array নাও
- শেষ index থেকে শুরুর দিকে গিয়ে প্রতিটা element নতুন array-তে রাখো

PHP

```
function reverseCopy(array $arr): array {
    $result = [];
    for ($i = count($arr) - 1; $i >= 0; $i--) {
        $result[] = $arr[$i];
    }
    return $result;
}
print_r(reverseCopy([1, 2, 3, 4, 5])); // [5,4,3,2,1]
```

Time: O(n)

Space: O(n)

কাজ করে, কিন্তু extra array লাগছে। প্রশ্ন "in-place" চাইলে এটা গ্রহণযোগ্য না।

পদ্ধতি ২: Two Pointer দিয়ে In-place Swap

1. $L = 0$, $R =$ শেষ index
2. যতক্ষণ $L < R$: $arr[L]$ আর $arr[R]$ swap করো
3. $L++$ আর $R--$

PHP

```
function reverseInPlace(array &$arr): void { // & মানে আসল array-টাই বদলাবে
    $left = 0;
    $right = count($arr) - 1;
    while ($left < $right) {
        [$arr[$left], $arr[$right]] = [$arr[$right], $arr[$left]]; // swap
        $left++;
        $right--;
    }
}

$a = [1, 2, 3, 4, 5];
reverseInPlace($a);
print_r($a); // [5,4,3,2,1]

// String-এর জন্য একই কৌশল (PHP string-এ index দিয়ে access করা যায়)
function reverseString(string $s): string {
    $l = 0; $r = strlen($s) - 1;
    while ($l < $r) {
        $tmp = $s[$l]; $s[$l] = $s[$r]; $s[$r] = $tmp;
        $l++; $r--;
    }
    return $s;
}

echo reverseString("hello"); // olleh
```

Time: $O(n)$ Space: $O(1)$ মোট $n/2$ বার swap হয় $\rightarrow O(n/2) = O(n)$

ড্রাই রান — ধাপে ধাপে দেখি

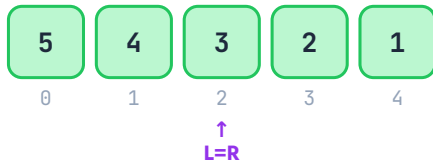
ধাপ 1 শুরু: $L=0$, $R=4$ । $arr[0]=1$ আর $arr[4]=5$ swap।



ধাপ 2 swap-এর পর। এখন $L=1$, $R=3$ । 2 আর 4 swap।



ধাপ 3 এখন $L=2$, $R=2$ । $L < R$ মিথ্যা $\rightarrow$ থামো। মার্কের 3 নিজের জায়গায় ঠিক আছে।



এজ কেস (এগুলো ভুলবে না)

- খালি array $[]$ $\rightarrow$ লুপ চলবেই না, ঠিক আছে
- একটা element $[7]$ $\rightarrow L=R=0$, কিছু করতে হয় না
- জোড় সংখ্যক element $[1,2,3,4]$ $\rightarrow L$ আর R পাশাপাশি এসে পার হয়ে যায়

ইন্টারভিউতে যা বলবে

- PHP-তে array ফাংশনে পাঠালে কপি হয় — in-place বদলাতে `&$arr` (reference) লাগে, এটা বললে ভালো ইমপ্রেশন পড়ে।
- Swap-এর PHP শর্টকাট: `[$a, $b] = [$b, $a];`

ফলো-আপ প্রশ্ন (ইন্টারভিউয়ার জিজ্ঞেস করতে পারে)

"একটা string palindrome কি না চেক করো" — একই Two Pointer: L আর R -এর অক্ষর মিলছে কি না দেখো, না মিললে false। (LeetCode 125, প্র্যাকটিসে আছে)

প্রশ্ন 1.3

Best Time to Buy and Sell Stock

শেয়ার কোন দিন কিনে কোন দিন বেচলে সর্বোচ্চ লাভ

Easy

LeetCode 121

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

"এখন পর্যন্ত সবচেয়ে কম মান মনে রাখো" — এই কৌশল অসংখ্য প্রশ্নে লাগে। খুব জনপ্রিয় ইন্টারভিউ প্রশ্ন।

সমস্যাটা কী?

`prices[i]` = *i*-তম দিনে একটা শেয়ারের দাম। তুমি একদিন কিনবে আর **পরের কোনো দিন** বিক্রি করবে। সর্বোচ্চ লাভ কত? লাভ না হলে 0।

উদাহরণ

Input: [7, 1, 5, 3, 6, 4]

Output: 5 // দিন 1-এ 1 টাকায় কিনে দিন 4-এ 6 টাকায় বেচো

Input: [7, 6, 4, 3, 1]

Output: 0 // দাম শুধু কমছে, কিনলেই লস

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

তুমি প্রতিদিন বাজারে আলুর দাম দেখছো। প্রতিদিন নিজেকে জিজ্ঞেস করো: "আজ পর্যন্ত সবচেয়ে সস্তা দিনে কিনে থাকলে, আজ বেচলে লাভ কত?" সবচেয়ে সস্তা দামটা একটা খাতায় লিখে রাখলেই হয় — পেছনে বারবার দেখতে হয় না।

পদ্ধতি ১: Brute Force — প্রতিটা কেনা-বেচার জোড়া

1. প্রতিটা কেনার দিন *i*
2. তার পরের প্রতিটা বেচার দিন *j*
3. `prices[j] - prices[i]` সর্বোচ্চটা রাখো

PHP

```
function maxProfitBrute(array $prices): int {
    $best = 0;
    $n = count($prices);
    for ($i = 0; $i < $n; $i++) {
        for ($j = $i + 1; $j < $n; $j++) {
            $best = max($best, $prices[$j] - $prices[$i]);
        }
    }
    return $best;
}
echo maxProfitBrute([7, 1, 5, 3, 6, 4]); // 5
```

Time: $O(n^2)$ Space: $O(1)$

পদ্ধতি ২: Optimized — এক পাশে সর্বনিম্ন দাম মনে রাখা

1. `minPrice` = এখন পর্যন্ত দেখা সবচেয়ে কম দাম
2. প্রতিদিন: আজকের লাভ = আজকের দাম - `minPrice`
3. সর্বোচ্চ লাভ আপডেট করো
4. আজকের দাম কম হলে `minPrice` আপডেট করো

PHP

```
function maxProfit(array $prices): int {
    $minPrice = PHP_INT_MAX; // শুরুতে "অসীম"
    $best = 0;
    foreach ($prices as $price) {
        if ($price < $minPrice) {
            $minPrice = $price; // নতুন সস্তা দিন পেলাম
        } elseif ($price - $minPrice > $best) {
            $best = $price - $minPrice; // আজ বেচলে আরও বেশি লাভ
        }
    }
    return $best;
}
echo maxProfit([7, 1, 5, 3, 6, 4]); // 5
echo "\n" . maxProfit([7, 6, 4, 3, 1]); // 0
```

Time: $O(n)$ Space: $O(1)$

ড্রাই রান — ধাপে ধাপে দেখি

দিন	দাম	minPrice	আজ বেচলে লাভ	best
0	7	7	—	0
1	1	1 (নতুন min)	—	0
2	5	1	5-1 = 4	4
3	3	1	3-1 = 2	4
4	6	1	6-1 = 5	5
5	4	1	4-1 = 3	5

সবুজ = কেনার দিন (সবচেয়ে কম), নীল = বেচার দিন



এজ কেস (এগুলো ভুলবে না)

- দাম শুধু কমছে → লাভ 0 (নেগেটিভ না)
- একটা মাত্র দিন → 0
- খালি array → 0

ইন্টারভিউতে যা বলবে

- ভুল ধারণা ধরিয়ে দাও: "সর্বোচ্চ - সর্বনিম্ন" না! কারণ সর্বোচ্চ দাম সর্বনিম্নের আগে আসতে পারে ([5, 1] → উত্তর 0, 4 না)।

ফলো-আপ প্রশ্ন (ইন্টারভিউয়ার জিজ্ঞেস করতে পারে)

"একাধিকবার কেনা-বেচা করা গেলে?" (LeetCode 122) — প্রতিবার দাম বাড়লে ($\text{prices}[i] > \text{prices}[i-1]$) সেই পার্থক্য যোগ করো।

প্রশ্ন 1.4

Move Zeroes

সব শূন্য অ্যারের শেষে পাঠাও

Easy

LeetCode 283

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

Slow-Fast Two Pointer-এর ক্লাসিক উদাহরণ। "in-place" আর "ক্রম ঠিক রাখা" — দুটো শর্ত একসাথে সামলানো শেখায়।

সমস্যাটা কী?

একটা array দেওয়া আছে। সব 0-কে শেষে পাঠাও, কিন্তু বাকি সংখ্যাগুলোর **আপেক্ষিক ক্রম** একই থাকবে। কাজটা in-place করতে হবে।

উদাহরণ

Input: [0, 1, 0, 3, 12]

Output: [1, 3, 12, 0, 0]

Input: [0]

Output: [0]

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

একটা শেলফে কিছু খালি বাক্স (0) আর কিছু ভরা বাক্স আছে। তুমি বাম থেকে হাঁটছো (fast)। তোমার বন্ধু দাঁড়িয়ে আছে "পরের ভরা বাক্সটা এখানে বসবে" জায়গায় (slow)। ভরা বাক্স পেলেই বন্ধুর জায়গায় পাঠাও, বন্ধু এক ঘর এগোয়।

পদ্ধতি ১: আলাদা array ব্যবহার

- প্রথমে সব non-zero নতুন array-তে রাখো
- বাকি জায়গা 0 দিয়ে ভরো

PHP

```
function moveZeroesExtra(array $nums): array {
    $result = [];
    foreach ($nums as $x) {
        if ($x !== 0) $result[] = $x;
    }
    while (count($result) < count($nums)) {
        $result[] = 0;
    }
    return $result;
}
print_r(moveZeroesExtra([0, 1, 0, 3, 12])); // [1,3,12,0,0]
```

Time: O(n)

Space: O(n)

in-place না, তাই প্রশ্নের শর্ত মানে না।

পদ্ধতি ২: Slow-Fast Two Pointer (in-place)

1. `slow = 0` — পরের non-zero কোথায় বসবে
2. `fast` দিয়ে পুরো array ঘুরো
3. `nums[fast] ≠ 0` হলে `nums[slow]` আর `nums[fast]` swap করো, `slow++`

PHP

```
function moveZeroes(array &$nums): void {
    $slow = 0;
    $n = count($nums);
    for ($fast = 0; $fast < $n; $fast++) {
        if ($nums[$fast] !== 0) {
            [$nums[$slow], $nums[$fast]] = [$nums[$fast], $nums[$slow]];
            $slow++;
        }
    }
}

$a = [0, 1, 0, 3, 12];
moveZeroes($a);
print_r($a); // [1,3,12,0,0]
```

Time: $O(n)$

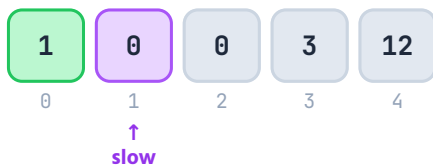
Space: $O(1)$

ড্রাই রান — ধাপে ধাপে দেখি

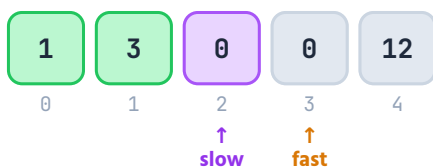
ধাপ 1 শুরু: `slow=0`, `fast=0`। `nums[0]=0` → কিছু করি না।



ধাপ 2 `fast=1`, `nums[1]=1 ≠ 0` → `swap(0,1)`, `slow=1`



ধাপ 3 `fast=2`, `nums[2]=0` → কিছু না। `fast=3`, `nums[3]=3` → `swap(1,3)`, `slow=2`



ধাপ 4 fast=4, nums[4]=12 → swap(2,4), slow=3। শেষ!



এজ কেস (এগুলো ভুলবে না)

- কোনো 0 নেই [1,2,3] → প্রতিবার নিজের সাথেই swap, array একই থাকে
- সব 0 [0,0,0] → কোনো swap হয় না
- এক element

ইন্টারভিউতে যা বলবে

- বলো: "slow-এর বামের সবকিছু non-zero আর সঠিক ক্রমে — এটাই আমার invariant (সবসময় সত্য থাকা শর্ত)।"

প্রশ্ন 1.5

Remove Duplicates from Sorted Array

Sorted array থেকে ডুপ্লিকেট বাদ দাও (in-place)

Easy

LeetCode 26

কেন এটা গুরুত্বপূর্ণ?

Slow-Fast pointer-এর আরেকটা ব্যবহার। "sorted" শব্দটা দেখে বুঝতে হবে ডুপ্লিকেটগুলো পাশাপাশি থাকে।

সমস্যাটা কী?

একটা **sorted** array থেকে in-place ডুপ্লিকেট সরাও, যাতে প্রতিটা মান একবার থাকে। ইউনিক element-এর সংখ্যা k রিটার্ন করো। প্রথম k টা ঘরে ইউনিক মানগুলো থাকবে (বাকি ঘরে কী আছে তাতে কিছু যায় আসে না)।

উদাহরণ

Input: [1, 1, 2, 2, 3]

Output: 3, nums = [1, 2, 3, _, _]

Input: [0,0,1,1,1,2,2,3,3,4]

Output: 5, nums = [0,1,2,3,4, ...]

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

sorted array মানে একই মানগুলো একসাথে দলবদ্ধ। তাই কোনো মান নতুন কি না বুঝতে শুধু "শেষ রাখা মানের" সাথে তুলনা করলেই হয়।

পদ্ধতি ১: আলাদা array-তে ইউনিক রাখা

1. নতুন array নাও
2. প্রতিটা element আগের রাখা শেষ element থেকে আলাদা হলে রাখো

PHP

```
function uniqueExtra(array $nums): array {
    $res = [];
    foreach ($nums as $x) {
        if (empty($res) || end($res) !== $x) { // end() = শেষ element
            $res[] = $x;
        }
    }
    return $res;
}
print_r(uniqueExtra([1, 1, 2, 2, 3])); // [1,2,3]
```

Time: $O(n)$ Space: $O(n)$

পদ্ধতি ২: Slow-Fast Pointer (in-place)

1. $k = 1$ (প্রথম element সবসময় ইউনিক)
2. $i = 1$ থেকে: $\text{nums}[i] \neq \text{nums}[k-1]$ হলে নতুন মান $\rightarrow \text{nums}[k] = \text{nums}[i]$, $k++$
3. শেষে k রিটার্ন

PHP

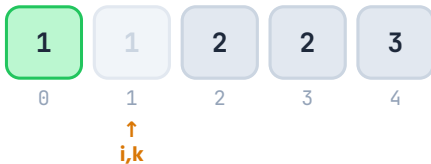
```
function removeDuplicates(array &$nums): int {
    $n = count($nums);
    if ($n === 0) return 0;
    $k = 1; // পরের ইউনিক কোথায় বসবে
    for ($i = 1; $i < $n; $i++) {
        if ($nums[$i] !== $nums[$k - 1]) { // শেষ রাখা মান থেকে আলাদা?
            $nums[$k] = $nums[$i];
            $k++;
        }
    }
    return $k;
}

$a = [1, 1, 2, 2, 3];
$k = removeDuplicates($a);
echo $k . " → " . implode(",", array_slice($a, 0, $k)); // 3 → 1,2,3
```

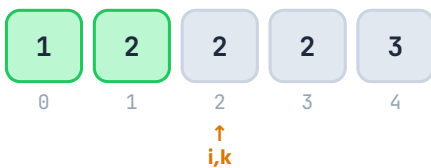
Time: $O(n)$ Space: $O(1)$

ড্রাই রান — ধাপে ধাপে দেখি

ধাপ 1 শুরু: $k=1$ । $i=1$: $\text{nums}[1]=1$, $\text{nums}[k-1]=1 \rightarrow$ একই, skip



ধাপ 2 $i=2$: $\text{nums}[2]=2 \neq 1 \rightarrow \text{nums}[1]=2$, $k=2$



ধাপ 3 $i=3$: $2 == \text{nums}[1]=2 \rightarrow$ skip



ধাপ 4 $i=4: 3 \neq 2 \rightarrow \text{nums}[2]=3, k=3$ । শেষ। প্রথম 3 ঘর = [1,2,3]



এজ কেস (এগুলো ভুলবে না)

- খালি array $\rightarrow 0$
- সব একই [5,5,5] $\rightarrow 1$
- কোনো ডুপ্লিকেট নেই $\rightarrow n$

ইন্টারভিউতে যা বলবে

- PHP-তে `array_unique()` আছে, কিন্তু সেটা in-place না এবং index ঠিক রাখে না — ইন্টারভিউতে নিজে লিখে দেখাও।

থিওরি প্রশ্নোত্তর (ইন্টারভিউ স্টাইল)

প্রতিটা উত্তর ইন্টারভিউতে ৩০-৬০ সেকেন্ডে বলার মতো করে লেখা। মুখস্থ না করে নিজের ভাষায় বলার প্র্যাকটিস করো।

প্রশ্ন 1: Big-O Notation কী?

Big-O বলে ইনপুটের সাইজ (n) বাড়লে একটা অ্যালগরিদমের কাজ বা সময় কী হারে বাড়ে — সাধারণত worst case ধরে। এটা আসল সেকেন্ড মাপে না, বরং growth rate মাপে, তাই যেকোনো মেশিনে তুলনা করা যায়। যেমন $O(n)$ মানে ইনপুট দ্বিগুণ হলে সময়ও মোটামুটি দ্বিগুণ।

প্রশ্ন 2: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$ -এর একটা করে উদাহরণ দাও।

Complexity	উদাহরণ
$O(1)$	Array-তে index দিয়ে access, hash map lookup
$O(\log n)$	Binary search
$O(n)$	Linear search, array-র যোগফল
$O(n \log n)$	Merge sort, PHP-এর <code>sort()</code>
$O(n^2)$	Bubble sort, nested লুপ দিয়ে সব জোড়া চেক

প্রশ্ন 3: Big-O হিসাবে constant কেন বাদ দিই?

কারণ n অনেক বড় হলে growth rate-টাই আসল পার্থক্য তৈরি করে। $2n$ আর n দুটোই লিনিয়ার — ইনপুট দ্বিগুণ হলে দুটোরই সময় দ্বিগুণ। কিন্তু n^2 আর n -এর পার্থক্য n বাড়ার সাথে বিশাল হয়।

প্রশ্ন 4: Best, Average আর Worst case কী?

Linear search-এ টার্গেট প্রথম ঘরে থাকলে best case $O(1)$, গড়ে মাঝামাঝি পাওয়া যায় (average $O(n)$), শেষে বা না থাকলে worst case $O(n)$ । ইন্টারভিউতে সাধারণত worst case বলি কারণ এটা একটা গ্যারান্টি।

প্রশ্ন 5: Time আর Space complexity-র মধ্যে trade-off বলতে কী বোঝো?

অনেক সময় বেশি মেমোরি ব্যবহার করে দ্রুত সলিউশন পাওয়া যায়। যেমন Two Sum: brute force $O(n^2)$ time, $O(1)$ space; hash map দিয়ে $O(n)$ time কিন্তু $O(n)$ space। কোনটা নেব তা নির্ভর করে কোনটার সীমাবদ্ধতা বেশি — সাধারণত time বেশি গুরুত্বপূর্ণ।

প্রশ্ন 6: Array-তে index দিয়ে access $O(1)$ কিন্তু মাঝে insert $O(n)$ কেন?

Array-র element মেমোরিতে পাশাপাশি থাকে, তাই start address + index $\times$ size দিয়ে সরাসরি যেকোনো ঘরে যাওয়া যায় — $O(1)$ । কিন্তু মাঝে insert করতে হলে তার পরের সব element এক ঘর সরাতে হয় — worst case n টা, তাই $O(n)$ ।

প্রশ্ন 7: PHP array আর C/Java-র array-র পার্থক্য কী?

PHP array আসলে ordered hash map — key integer বা string হতে পারে, সাইজ ডায়নামিক, একসাথে list ও dictionary হিসেবে কাজ করে। C/Java array-র সাইজ fixed, একই টাইপের element, মেমোরিতে পাশাপাশি। এজন্য PHP-তে `isset($arr[$key])` $O(1)$ এ key খুঁজতে পারে।

প্রশ্ন 8: Two Pointer টেকনিক কী এবং কখন ব্যবহার করবে?

দুইটা index দিয়ে একসাথে array-তে চলা, যাতে nested লুপ ছাড়া এক পাসে কাজ হয়। দুই ধরন: (১) দুই মাথা থেকে ভেতরে — reverse, palindrome, sorted array-তে pair sum; (২) slow-fast — in-place filter, duplicate বাদ দেওয়া। Sorted array বা in-place শর্ত থাকলে প্রথমে এটা ভাবি।

প্রশ্ন 9: Space complexity হিসাবে ইনপুট কি ধরা হয়?

সাধারণত আমরা "auxiliary space" বলি — মানে ইনপুট বাদে অতিরিক্ত যা লাগে। শুধু কয়েকটা ভেরিয়েবল থাকলে $O(1)$ । তবে ইন্টারভিউয়ারকে স্পষ্ট করে বলা ভালো যে তুমি auxiliary space বলছো।

প্র্যাকটিস প্রবলেম (নিজে সলভ করো)

আগে নিজে ২০ মিনিট চেষ্টা করো। আটকে গেলে হিন্ট দেখো। তারপরও না পারলে সলিউশন দেখে, পরদিন আবার নিজে লেখো।

1. Valid Palindrome (Easy · LeetCode 125)

একটা string (শুধু অক্ষর আর সংখ্যা ধরে, case ignore করে) palindrome কি না বলো। "A man, a plan, a canal: Panama" → true

হিন্ট: Two pointer L, R। অক্ষর/সংখ্যা না হলে skip করো (ctype_alnum)। strtolower দিয়ে তুলনা।

2. Two Sum II — Sorted Array (Medium · LeetCode 167)

Sorted array-তে দুইটা সংখ্যা খোঁজো যাদের যোগফল target, O(1) space-এ।

হিন্ট: L=0, R=শেষ। sum < target → L++, sum > target → R--।

3. Squares of a Sorted Array (Easy · LeetCode 977)

[-4,-1,0,3,10] → প্রতিটার বর্গ, sorted ক্রমে: [0,1,9,16,100]

হিন্ট: সবচেয়ে বড় বর্গ দুই মাথার কোনো একটায় থাকে। দুই মাথা থেকে তুলনা করে result-এর শেষ থেকে ভরো।

4. Maximum Subarray (Kadane) (Medium · LeetCode 53)

পাশাপাশি element-এর কোন অংশের যোগফল সর্বোচ্চ? [-2,1,-3,4,-1,2,1,-5,4] → 6

হিন্ট: current = max(num, current + num)। আগের যোগফল নেগেটিভ হলে ফেলে দিয়ে নতুন করে শুরু করো।

5. Find Largest and Second Largest (Easy)

একটা array-তে সবচেয়ে বড় আর দ্বিতীয় বড় সংখ্যা এক পাসে বের করো (sort ছাড়া)।

হিন্ট: দুইটা ভেরিয়েবল first, second। নতুন সংখ্যা first-এর চেয়ে বড় হলে second = first, first = নতুন।

6. Running Sum of 1d Array (Easy · LeetCode 1480)

[1,2,3,4] → [1,3,6,10]

হিন্ট: nums[i] += nums[i-1] — i=1 থেকে।

চিট শিট — দিন ১

Complexity এক নজরে

কোড প্যাটার্ন	Time
একটা লুপ	$O(n)$
লুপের ভেতর লুপ	$O(n^2)$
প্রতিবার অর্ধেক	$O(\log n)$
পরপর দুইটা লুপ	$O(n)$
sort() তারপর লুপ	$O(n \log n)$
isset(\$map[\$k])	$O(1)$

আজকের প্যাটার্নগুলো

প্রশ্ন	কৌশল	মূল লাইন	Time / Space
Two Sum	Hash map	<code>need = target - num ; isset(\$seen[\$need])</code>	$O(n) / O(n)$
Reverse	Two pointer (দুই মাথা)	<code>swap L, R; L++, R--</code>	$O(n) / O(1)$
Stock	চলমান minimum	<code>best = max(best, price - minPrice)</code>	$O(n) / O(1)$
Move Zeroes	Slow-fast	non-zero হলে <code>swap(slow, fast)</code> , <code>slow++</code>	$O(n) / O(1)$
Remove Duplicates	Slow-fast	<code>nums[i] != nums[k-1]</code> হলে <code>nums[k++] = nums[i]</code>	$O(n) / O(1)$

PHP মনে রাখার জিনিস

- in-place পরিবর্তনের জন্য `function f(array &$a)`
- Swap: `[$a[$i], $a[$j]] = [$a[$j], $a[$i]];`
- Integer division: `intdiv($a, $b)`
- অসীম: `PHP_INT_MAX`, `PHP_INT_MIN`
- তুলনায় `===` ব্যবহার করো (টাইপসহ মেলায়)

আজকের নিজেকে যাচাই

- ☐ Two Sum না দেখে লিখতে পারি
- ☐ যেকোনো লুপ দেখে Big-O বলতে পারি
- ☐ Two pointer-এর দুই ধরন উদাহরণসহ বলতে পারি
- ☐ Array insert কেন $O(n)$ ব্যাখ্যা করতে পারি